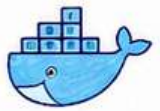


1. DOCKER FUNDAMENTALS

Build, Ship and Run Any App, Anywhere



What is Docker?

- Docker is a platform that uses **containerization** to package an application and all its dependencies so it **can run consistently** on any environment.
- It solves "works on my machine" problem.
- Lightweight, portable, isolated, scalable.

Key Benefits

- ✓ Consistency
- ✓ Portability
- ✓ Isolation
- ✓ Efficiency
- ✓ Scalability
- ✓ Faster Delivery

Use Cases

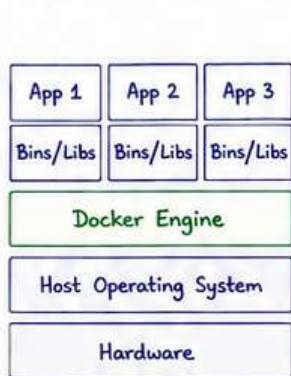
- Microservices
- CI/CD pipelines
- Dev & Test Environments
- Cloud & Hybrid Deployments
- Legacy App Modernization

Virtualization vs Containers

VIRTUAL MACHINES (VMs)



CONTAINERS (Docker)



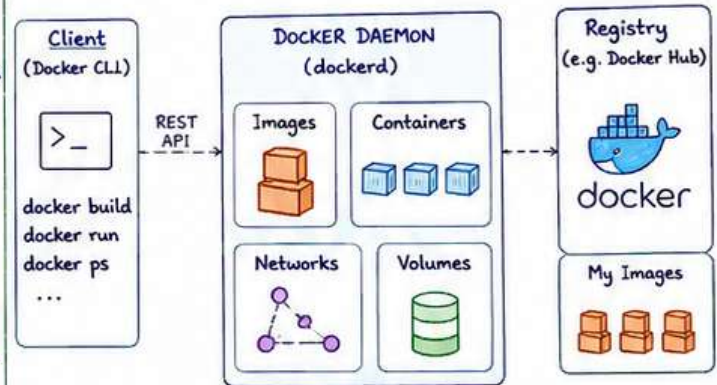
- VMs virtualize hardware.
- Containers virtualize OS.
- VMs are heavy, containers are lightweight.
- VMs boot in minutes.
- Containers start in seconds.
- Containers share host OS kernel.

* Containers = Processes with isolated filesystem, network and resources on a shared OS kernel.

Core Docker Concepts

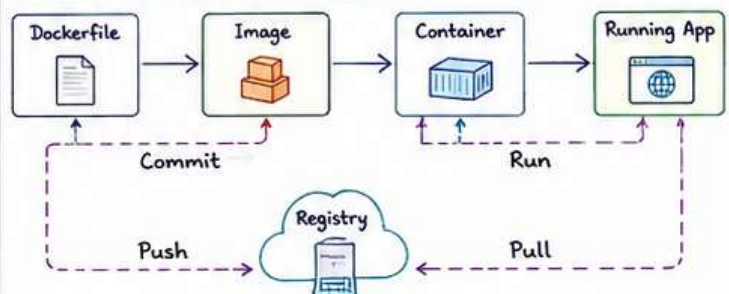
- * **Image**: A read-only template with instructions to create a container. (Built from Dockerfile)
- * **Container**: A runnable instance of an image.
- * **Dockerfile**: A text file with instructions to build an image.
- * **Repository**: A collection of related images (e.g. on Docker Hub).
- * **Tag**: A label for an image (e.g. nginx:1.25).
- * **Registry**: A storage and distribution system for Docker images.
- * **Volume**: Managed storage for persistent data.
- * **Network**: Virtual network for containers to communicate.

Docker Architecture (High Level)



- **Client**: You interact using Docker CLI (or API).
- **Daemon (dockerd)**: The background service that builds, runs and manages Docker objects.
- **Images**: Read-only templates used to create containers.
- **Containers**: Running instances of images.
- **Networks**: Enable communication between containers and external world.
- **Volumes**: Persist data beyond container lifecycle.
- **Registry**: Stores and distributes Docker images.

Docker Objects Relationship



Why Docker is Important?

- 👍 Eliminates environment inconsistencies.
- 👍 Enables rapid development and testing.
- 👍 Simplifies application deployment.
- 👍 Supports microservices architecture.
- 👍 Runs anywhere - laptop, server, cloud.



Docker Ecosystem (some tools)

Docker Engine
(Core Runtime)



Docker CLI
(Command line interface)



Docker Compose
(Multi-container apps)



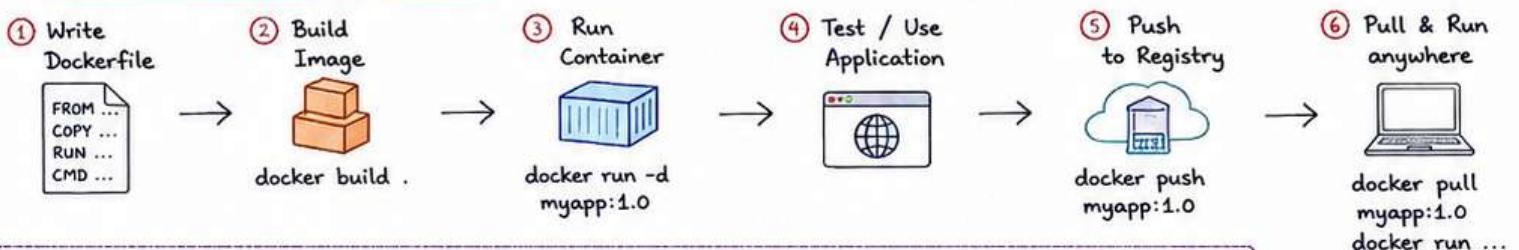
Docker Hub
(Public Registry)



Docker Desktop
(GUI for Mac/Win)



Quick Docker Workflow (Summary)



* Docker helps you containerize your application once and run it consistently everywhere! ❤️

2. INSTALLATION, DOCKER CLI, IMAGES, CONTAINERS LIFECYCLE



Docker runs on Linux, Windows, macOS 😊

1) INSTALLATION (Overview)

Docker Desktop (for Mac/Windows)

- 1) Download Docker Desktop from docker.com/products/docker-desktop
- 2) Install and run the installer.
- 3) Start Docker Desktop.
- 4) Verify Installation:

```
docker --version
docker run hello-world
```



Docker Engine (for Linux)

- 1) Uninstall old versions
`sudo apt-get remove docker docker-engine docker.io containerd runc`
- 2) Install using repository

```
sudo apt update
sudo apt install ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo $VERSION_CODENAME) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

- 3) Start & enable
`sudo systemctl start docker`
`sudo systemctl enable docker`
- 4) Verify: `docker run hello-world`

2) DOCKER CLI ESSENTIALS

Docker CLI communicates with Docker Daemon (dockerd).

Category	Command	Example	Description
System	<code>docker version</code>		Shows Docker version info
	<code>docker info</code>		Shows system-wide info
	<code>docker system df</code>		Shows disk usage
Images	<code>docker images</code>		Lists images
	<code>docker pull <image></code>	<code>docker pull nginx:latest</code>	Pull image from registry
	<code>docker rmi <image></code>	<code>docker rmi nginx</code>	Remove image
Containers	<code>docker ps</code>	<code>docker ps -a</code>	List running containers
	<code>docker run <image></code>	<code>docker run -it ubuntu</code>	Create & start container
	<code>docker start <id></code>		Start stopped container
	<code>docker stop <id></code>		Stop running container
	<code>docker restart <id></code>		Restart container
	<code>docker rm <id></code>		Remove stopped container
	<code>docker logs <id></code>	<code>docker logs -f <id></code>	View container logs
	<code>docker exec -it <id> sh</code>		Execute command in container
Other	<code>docker inspect <id></code>		Detailed info (JSON)
	<code>docker cp <src> <dst></code>		Copy files/folders
	<code>docker stats</code>		Live container stats
	<code>docker events</code>		Real-time events

★ Tip: Use TAB for auto-complete

3) IMAGES

- Immutable, read-only templates.
- Built from Dockerfile or pulled from registry.
- Made of layers (each instruction creates a layer).
- Stored in local image cache.

Common commands:

- `docker images`
- `docker pull <image>:<tag>`
- `docker search <keyword>`
- `docker rmi <image>`
- `docker history <image>`
↳ shows layers

IMAGE NAMING

`<registry>/<repository>:<tag>`
Examples:
`nginx:latest`
`ubuntu:22.04`
`docker.io/library/nginx:1.25`
`myregistry.com/app/web:v1`

Image (e.g, `nginx:1.25`)

Layer	Command	Notes
Layer 4	<code>CMD ["nginx", "-g", "daemon off;"]</code>	Top (writable)
Layer 3	<code>COPY nginx.conf /etc/nginx/</code>	
Layer 2	<code>RUN apk add --no-cache nginx</code>	
Layer 1	<code>FROM alpine:3.19</code>	Base (read-only)

- Layers are cached & shared → saves time & storage.

5) HANDS-ON EXAMPLES

- 1) Run nginx in background and map port
`docker run -d --name web -p 8080:80 nginx`
Access → `http://localhost:8080`

- 2) Run Ubuntu interactively
`docker run -it --name ubuntu1 ubuntu:22.04 bash`

- 3) List running containers
`docker ps`

- 4) Stop & remove container
`docker stop web`
`docker rm web`

- 5) Remove unused images
`docker image prune -a`

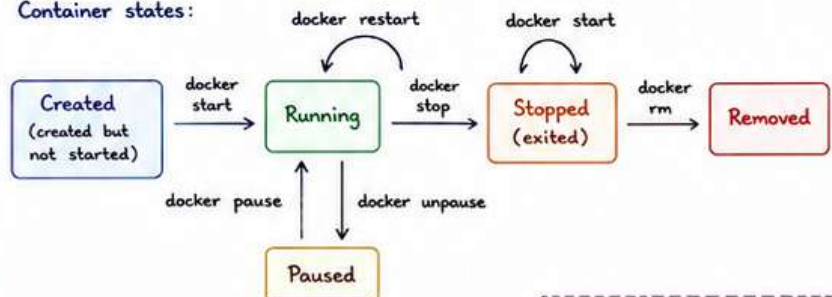
Tips

- ✓ Use `--rm` for ephemeral containers
- ✓ Use meaningful names



4) CONTAINER LIFECYCLE

Container states:



- `docker create <image>` → create container (not start)
- `docker start <idname>` → start created container
- `docker stop <idname>` → graceful stop (SIGTERM)
- `docker kill <idname>` → force stop (SIGKILL)
- `docker rm <idname>` → remove stopped container
- `docker run` = create + start

Useful flags with `docker run`

- `-d` Run in background
- `-it` Interactive (terminal)
- `--name <name>` Name container
- `-p host:container` Port mapping
- `-v volume:container` Volume mount
- `--rm` Auto remove on exit
- `-e KEY=VALUE` Environment vars

Check Status

- `docker ps` → running containers
- `docker ps -a` → all containers
- `docker logs -f <id>` → live logs
- `docker inspect <id>` → detailed JSON info

CONTAINER IDENTIFIERS

- You can use Container ID (full/short) or Name in most commands.
- Short ID is usually enough (first 12 chars).

Example:

`a1b2c3d4e5f6` (full)
`a1b2c3d4e5f6` (short)



SUMMARY

- ✓ Images are templates.
- ✓ Containers are running instances of images.
- ✓ Docker CLI helps manage images, containers & more.
- ✓ Container lifecycle: create → start → run → stop → remove.
- ✓ Master these basics before moving to advanced topics! ❤️



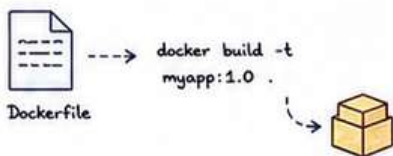
3. DOCKERFILE DEEP DIVE, LAYERS, CACHING, MULTI-STAGE BUILDS



Dockerfile is a blueprint to build docker images 😊

1) DOCKERFILE BASICS

A Dockerfile is a text file with instructions to build an image.



General Syntax

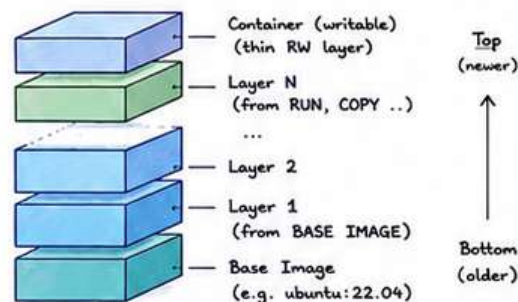
```
INSTRUCTION arguments
# comments
```

2) COMMON INSTRUCTIONS

FROM base image (must be first)
RUN run commands in new layer
COPY copy files from host to image
ADD copy + auto extract tar + remote URL
WORKDIR set working directory
ENV set environment variables
EXPOSE document port(s) the container listens on
CMD default command (overridden by docker run cmd)
ENTRYPOINT configure a container that will run as an executable
ARG build-time variables (not in final image by default)
LABEL add metadata to image
VOLUME declare mount point (anonymous volume)
USER set user to run subsequent instructions
HEALTHCHECK check container health
STOPSIGNAL set system call for container stop

3) IMAGE LAYERS (HOW IT WORKS)

- Each instruction creates a new **READ-ONLY** layer.
- Layers are stacked. Top layer is writable (container layer).
- Layers are cached and shared between images.



★ Changes in lower layers = rebuild from that layer onward.
Order of instructions matters!

4) DOCKERFILE EXAMPLE (SIMPLE NODE.JS APP)

```
1 # syntax=docker/dockerfile:1
2 FROM node:18-alpine # 1
3 WORKDIR /app # 2
4 COPY package*.json ./ # 3
5 RUN npm install --production # 4
6 COPY . . # 5
7 ENV NODE_ENV=production # 6
8 EXPOSE 3000 # 7
9 CMD ["node", "server.js"] # 8
```

What happens:

- Start from node:18-alpine
- Set /app as working dir
- Copy package files
- Install deps (new layer)
- Copy app source
- Set env var
- Document port 3000
- Default command

Each step = one layer
(if cache invalidated, rebuild from that step)

5) CACHING (HOW IT WORKS)

- Docker caches layers based on the instruction + context.
- If nothing changes, cached layer is reused → faster builds.
- Cache is invalidated when:
 - Instruction or arguments change
 - Files used by COPY/ADD change
 - Previous layer changed

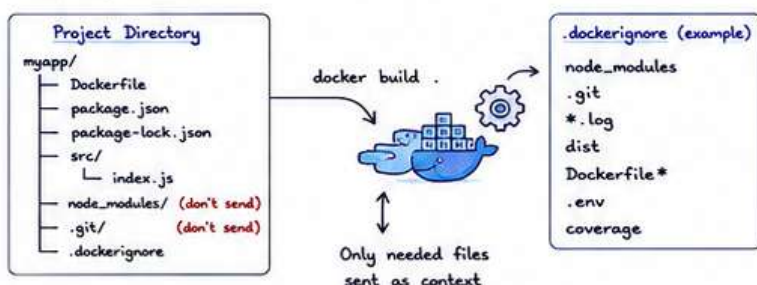
Step	Instruction	Changed?	Cache Hit?	Result
1	FROM node:18-alpine	No	✓	Reuse layer
2	WORKDIR /app	No	✓	Reuse
3	COPY package*.json ./	No	✓	Reuse
4	RUN npm install --prod	No	✓	Reuse
5	COPY . .	Yes	✗	Rebuild from here
6	ENV NODE_ENV=production	Yes	✗	Rebuild
7	EXPOSE 3000	Yes	✗	Rebuild
8	CMD ["node", "server.js"]	Yes	✗	Rebuild

★ TIP: Put less frequently changed steps (like installing deps) before frequently changed steps (like app source).



6) BUILD CONTEXT

- The build context is the set of files sent to the Docker daemon.
- By default: current directory (.)
- Use `.dockerignore` to exclude files, speed up builds, and keep images lean.



8) ARG vs ENV

ARG (build-time)

- Used during build
- Not persisted in final image (unless set to ENV)
- Good for passing variables like versions, tokens

Example:

```
FROM NODE_VERSION=18
FROM node:${NODE_VERSION}-alpine
```

ENV (run-time)

- Persisted in final image
- Available in container
- Good for app config

Example:

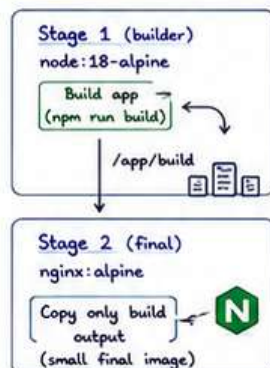
```
ENV NODE_ENV=production
ENV PORT=3000
```

7) MULTI-STAGE BUILDS

- Use multiple **FROM** statements.
- Artifacts from earlier stages can be copied to later stages.
- Produces smaller, cleaner final images.

```
# Stage 1 - Build
FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Stage 2 - Production
FROM nginx:alpine AS production
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```



★ Benefits: smaller image size, no build tools in final image, better security, faster deployments. ❤️

9) BEST PRACTICES

- ✓ Use official minimal base images (alpine, slim)
- ✓ Pin base image versions (FROM node:18.20-alpine)
- ✓ Order instructions to leverage cache
- ✓ Use `.dockerignore`
- ✓ Combine RUN commands with `&&` to reduce layers
- ✓ Remove unnecessary packages and caches
- ✓ Use multi-stage builds for smaller images
- ✓ Run as non-root user (USER node)
- ✓ Scan images for vulnerabilities



SUMMARY: Dockerfile defines image build steps. Each instruction creates a layer. Use caching smartly, build with clean context, and prefer multi-stage builds for efficient, secure images. 😊

4. VOLUMES, BIND MOUNTS, PERSISTENT STORAGE



Containers are ephemeral,
Data should be persistent!

1) WHY PERSISTENT STORAGE?

- Containers are ephemeral (created & destroyed).
- Data inside container's writable layer is lost when container is removed.
- Volumes & Bind mounts provide persistence.
- Useful for databases, logs, configs, uploads, etc.

Remember:
Ephemeral Container ≠ Ephemeral Data



2) STORAGE OPTIONS OVERVIEW

Type	Managed by	Location	Portability	Best for
Volumes	Docker	/var/lib/docker/volumes	High	DB data, logs, apps
Bind Mounts	Host	Any path on host	Low	Dev, config, code
tmpfs Mounts	Host (memory)	RAM	N/A (volatile)	Sensitive, temp data

3) DOCKER VOLUMES

- Managed by Docker.
- Stored in /var/lib/docker/volumes.
- Decoupled from container lifecycle.
- Easy to backup/migrate (with driver).

CREATE & USE VOLUME
docker volume create mydata
docker run -d --name db \
-v mydata:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=root \
mysql:8

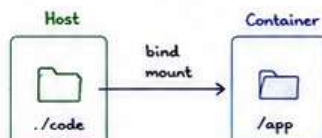
4) BIND MOUNTS

- Mount a file or directory from host into container.
- Reflects changes instantly.
- Depends on host path (less portable).
- Great for development & config files.

EXAMPLES

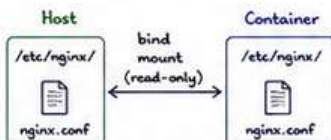
Mount host dir to container:

```
docker run -d --name app \  
-v $(pwd)/code:/app \  
-p 3000:3000 node:18
```



Mount single file:

```
docker run -d --name nginx \  
-v /etc/nginx/nginx.conf:/etc/nginx/nginx.conf:ro \  
-p 80:80 nginx
```



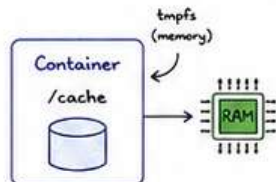
NOTE:

- Use :ro for read-only mount
- Use :rw (default) for read-write
- SELinux systems may need :z or :Z
:z = shared (multiple containers)
:Z = private (exclusive relabel)

6) tmpfs MOUNTS (IN-MEMORY)

- Stored in host memory (RAM).
- Super fast, but lost on restart.
- Good for sensitive/temp data (not written to disk).

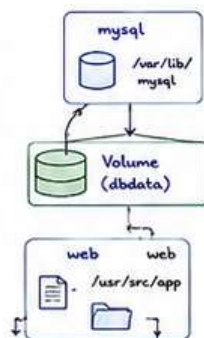
```
docker run --name cache \  
--tmpfs /cache:rw,size=100m \  
myapp
```



8) PRACTICAL EXAMPLES

A) Database with Volume

```
docker volume create dbdata  
docker run -d --name mysql \  
-v dbdata:/var/lib/mysql \  
-e MYSQL_ROOT_PASSWORD=root \  
-p 3306:3306 mysql:8
```



Data persists
even if container
is removed ✓

B) App with Bind Mount (Dev)

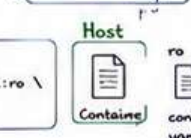
```
docker run -d --name web \  
-v $(pwd)/src:/usr/src/app \  
-p 8080:8080 node:18
```



Code changes
reflect instantly ✓

C) Read-only Config File

```
docker run -d --name app \  
-v /etc/app/config.yaml:/app/config.yaml:ro \  
myapp
```



Protected from
accidental change ✓

10) COMMON USE CASES

- ✓ Databases (MySQL, Postgres, MongoDB)
- ✓ Application state & uploads
- ✓ Log persistence
- ✓ Message queues (Kafka, RabbitMQ)
- ✓ Configuration files
- ✓ CI/CD artifacts



SUMMARY: Containers may come and go, but your data shouldn't.
Choose the right storage option based on portability, performance & use case.



5) VOLUME vs BIND MOUNT

Feature	Volume	Bind Mount
Managed by	Docker	Host
Location	/var/lib/docker/volumes	Any path on host
Lifecycle	Independent	Tied to host path
Portability	High	Low
Performance	Good	Very good (native)
Backup/Restore	Easy	Manual
Use case	Data persistence	Dev, config, source code

★ TIP: Use Volumes for data that must survive container/host recreation.
Use Bind Mounts for development & host file access.

7) INSPECT & MANAGE VOLUMES

List volumes

```
docker volume ls
```

Inspect a volume

```
docker volume inspect mydata
```

Remove a volume

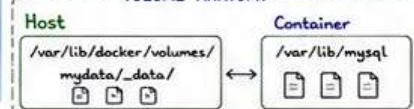
```
docker volume rm mydata
```

Prune unused volumes

```
docker volume prune
```

(careful! removes unused volumes)

VOLUME ANATOMY



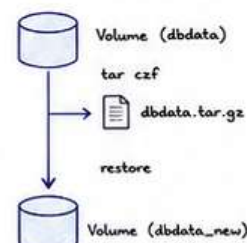
9) BACKUP & MIGRATION

Backup a Volume

```
docker run --rm -v dbdata:/data \  
-v $(pwd):/backup alpine \  
sh -c "cd /data && tar czf /backup/dbdata.tar.gz."
```

Restore to New Volume

```
docker volume create dbdata_new  
docker run --rm -v dbdata_new:/data \  
-v $(pwd):/backup alpine \  
sh -c "cd /data && tar xzf /backup/dbdata.tar.gz"
```



11) BEST PRACTICES

- ✓ Use volumes for production data.
- ✓ Use bind mounts for development.
- ✓ Avoid storing critical data in container writable layer.
- ✓ Always backup important volumes.
- ✓ Use read-only mounts for config/secrets when possible.
- ✓ Keep host paths consistent across environments.



5. DOCKER NETWORKING (BRIDGE, HOST, OVERLAY, DNS, PORTS)



Containers need to talk to each other and the outside world!

WHY NETWORKING?

- Containers are isolated by default.
- Networking enables communication between containers, host & external world.
- Docker provides built-in network drivers.



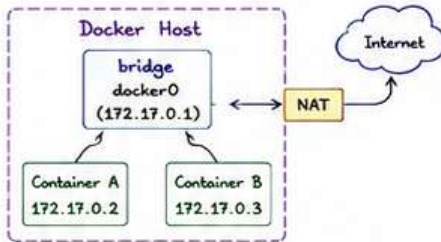
NETWORK CONCEPTS

- Container gets an IP from Docker network.
- Each network = isolated layer-2 domain.
- Containers on same network can talk using IP or name.
- By default, containers cannot access each other across different networks.

DOCKER NETWORK DRIVERS (Overview)

Driver	Scope	Use Case	Notes
bridge	Single Host	Default, isolated networks	NAT, virtual bridge
host	Single Host	High performance apps	No isolation (uses host net)
overlay	Multi-Host	Swarm / Multi-host comms	Encrypted, VXLAN
macvlan	Single Host	Containers appear as physical devices	Each gets MAC on LAN
none	Single Host	No networking	For custom network setup

1) BRIDGE NETWORK (Default)

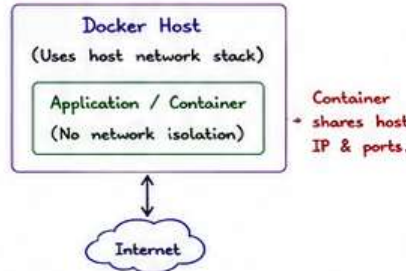


- Default network for standalone containers.
- Docker creates a virtual bridge (docker0).
- Containers get private IPs (usually 172.17.0.0/16).
- Outbound traffic → NATed to host IP.
- Communication: Containers on same bridge can talk directly.

Commands:

```
docker network ls
docker network create mybridge
docker run --network mybridge -d nginx
```

2) HOST NETWORK



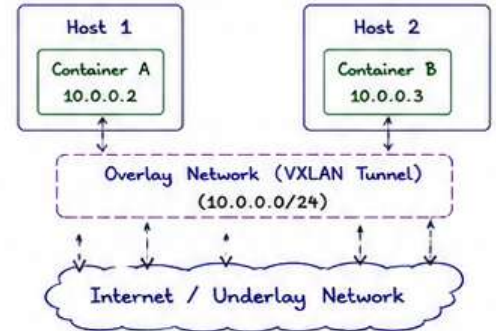
- Container uses host's network namespace.
- No NAT, no virtual network.
- Best performance (no network overhead).
- No port mapping needed.
- Not isolated - container can see all host network interfaces & ports.

Commands:

```
docker run --network host -d nginx
```

(Linux only)

3) OVERLAY NETWORK (Multi-Host)

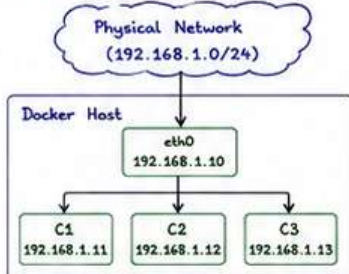


- Used in Docker Swarm for multi-host communication.
- Creates an encrypted overlay network across hosts.
- Containers act like they are on the same LAN.
- Uses VXLAN tunneling over UDP.

Commands (Swarm mode):

```
docker network create -d overlay myoverlay
docker service create --name app --network myoverlay nginx
```

4) MACVLAN NETWORK



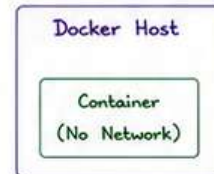
- Assigns a MAC address to container.
- Container appears as a physical device on the LAN.
- Useful for legacy apps, DHCP, monitoring.
- Container can communicate outside LAN directly (no NAT).

Commands:

```
docker network create -d macvlan \
--subnet=192.168.1.0/24 \
--gateway=192.168.1.1 \
-o parent=eth0 macnet
docker run -d --network macnet nginx
```

Note: Host cannot talk to macvlan container by default.

5) NONE NETWORK



- Container has no network interface.
- No IP, no connectivity.
- Used for custom network stacks or highly restricted containers.

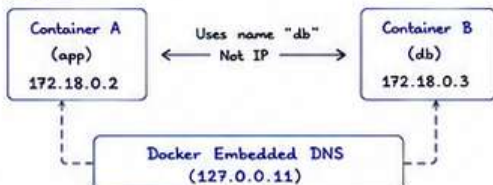
Command:

```
docker run --network none busybox
```

6) CONTAINER DNS & SERVICE DISCOVERY

- Docker provides embedded DNS server at 127.0.0.11.
- Containers resolve each other by name (on same network).
- Works for user-defined bridge, overlay networks.

Diagram:

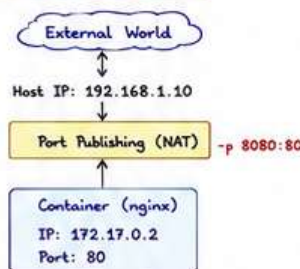


Test DNS:

```
docker exec -it app ping db
```

/etc/resolv.conf inside container points to 127.0.0.11

7) PORTS & PUBLISHING



- p hostPort:containerPort
- Access: `http://<hostIP>:<hostPort>`
- Multiple ports: `-p 8080:80 -p 8443:443`

Commands:

```
docker run -d -p 8080:80 nginx (All interfaces)
docker run -d -p 127.0.0.1:3306:3306 mysql (Localhost only)
```

8) COMMON NETWORK COMMANDS

<code>docker network ls</code>	List networks
<code>docker network inspect <net></code>	Inspect details
<code>docker network create <name></code>	Create network
<code>docker network rm <name></code>	Remove network
<code>docker network connect <net> <ctr></code>	Connect container
<code>docker network disconnect <net> <ctr></code>	Disconnect container
<code>docker network prune</code>	Remove unused networks

Inspect Example (bridge)

```
{
  "Name": "bridge",
  "Driver": "bridge",
  "Subnet": "172.17.0.0/16",
  "Gateway": "172.17.0.1",
  "Containers": {
    "abc123": { "IPv4Address": "172.17.0.2/16" }
  }
}
```



NETWORK FLOW (Bridge Example)



- Outbound: Container → Bridge → NAT → Host → Internet
- Inbound: Internet → Host IP:Port → NAT → Container IP:Port

BEST PRACTICES

- ✓ Use user-defined bridge networks.
- ✓ Name containers & use DNS (not IPs).
- ✓ Limit port exposure (bind to localhost if needed).
- ✓ Use overlay for multi-host (Swarm/K8s).
- ✓ Prefer least-privilege network access.



QUICK TIPS

- ★ Default bridge = quick & simple.
- ★ Host network = speed (no isolation).
- ★ Overlay = scale across hosts.
- ★ DNS name > IP address.
- ★ Expose only what you need!



SUMMARY: Docker networking gives containers connectivity & isolation. Choose the right driver for your needs, use DNS names, and publish ports securely.



6. DOCKER COMPOSE DEEP DIVE

Define multi-container applications
Run everything with a single command!



1) WHAT IS DOCKER COMPOSE?

- Tool for defining & running multi-container Docker applications.
- Configure app services, networks, volumes in a **YAML file** (compose.yml).
- Manage the full application lifecycle with simple commands.

Core Benefits

- ✓ Simple & Declarative
- ✓ Consistent Environments
- ✓ Easy Scaling
- ✓ Shared Networks & Volumes
- ✓ Great for Dev & CI/CD

2) COMPOSE FILE BASICS

Default file name: `compose.yml` (or `docker-compose.yml`)
Version (Compose Spec): `version: "3.9"` (or omit for latest)

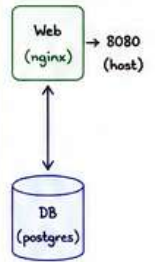
```
version: "3.9"
services: # All application services
  web: # Service name
    image: nginx:alpine # Or build:
    ports:
      - "8080:80" # Host:Container
    environment:
      - ENV=prod
    volumes:
      - ./html:/usr/share/nginx/html:ro
    depends_on: # Start order (not wait-for-ready)
      - db
  networks:
    app_net: # Custom network
  volumes:
    db_data: # Named volume
```

Top-level keys

- services → containers
- networks → networks
- volumes → named volumes
- configs → configs (swarm)
- secrets → secrets (swarm)

3) SIMPLE EXAMPLE

```
version: "3.9"
services:
  web:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./web:/usr/share/nginx/html:ro
    depends_on:
      - db
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: appdb
    volumes:
      - db_data:/var/lib/postgresql/data
  volumes:
    db_data:
  networks:
    default:
      name: app_net
```



4) KEY FEATURES & SECTIONS

Services

Define containers that make up the application.
build → Build image from Dockerfile
image → Use image from registry/local
command → Override default CMD
entrypoint → Override default ENTRYPOINT

Ports

"host:container" or "ip:host:container"

Volumes

- Named volume: `db_data:/var/lib/mysql`
- Bind mount: `./code:/usr/src/app`

Environment

- environment: KEY=VALUE
- env_file: .env

Depends_on

- Controls start order.
- Doesn't wait for service to be "ready".

Networks

- Default network is auto-created.
- You can define custom networks.

Restart Policies

restart: no | always | on-failure | unless-stopped

5) COMMON COMMANDS

Command	Description
<code>docker compose up</code>	Create & start all services
<code>docker compose up -d</code>	Run in background (detached)
<code>docker compose down</code>	Stop & remove containers, networks
<code>docker compose down -v</code>	Also remove named volumes
<code>docker compose ps</code>	List running services
<code>docker compose logs -f</code>	View logs (follow mode)
<code>docker compose exec <svc> sh</code>	Open shell in a service
<code>docker compose build</code>	Build images
<code>docker compose pull</code>	Pull latest images
<code>docker compose config</code>	Validate & view final config
<code>docker compose top</code>	Display running processes

★ Tip: Use `-d` for background, `-f <file>` to specify another compose file.

6) ADVANCED CONFIGURATION

a) Build Options

```
services:
  app:
    build:
      context: .
      dockerfile: Dockerfile
      args:
        APP_ENV: production
```

b) Environment File (.env)

```
# .env
DB_USER=admin
DB_PASS=secret
DEBUG=false
PORT=8080
```

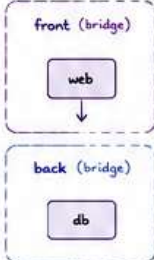
compose.yml
env_file: .env

c) Healthcheck

```
services:
  web:
    image: nginx
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost/"]
      interval: 10s
      timeout: 3s
      retries: 3
```

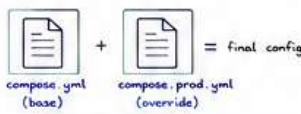
d) Resource Limits

```
networks:
  app:
    image: myapp
    deploy:
      resources:
        limits:
          cpus: "0.50"
          memory: 512M
```



f) Multiple Compose Files

`docker compose -f compose.yml -f compose.prod.yml up -d`



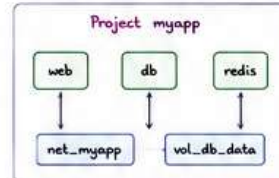
g) Profiles (optional services)

```
services:
  debug:
    image: alpine
    profiles: [debug]
```

Run with profile → `docker compose --profile debug`

7) PROJECTS

- Compose uses a **PROJECT** to isolate resources (networks, volumes, containers).
- Default project name = current directory name.
- Use `-p` to set a custom project name.



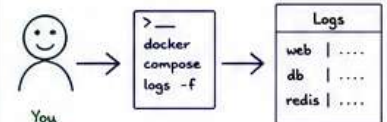
`docker compose -p shop up -d`

Useful:

- Avoids name conflicts
- Easy parallel environments
- Used in CI/CD pipelines

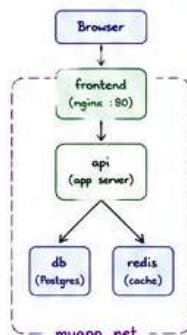
8) DEBUGGING & INSPECTION

- Final merged config:
`docker compose config`
- Service logs:
`docker compose logs <service>`
- Exec into container:
`docker compose exec <service> sh`
- Inspect service:
`docker compose ps`
- Show container resource usage:
`docker compose top`



9) REAL-WORLD EXAMPLE (3-TIER APP)

```
version: "3.9"
services:
  frontend:
    image: nginx:alpine
    ports:
      - "80:80"
    depends_on:
      - api
  api:
    build: ./api
    environment:
      DB_HOST: db
      REDIS_HOST: redis
    depends_on:
      - db
      - redis
  db:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: secret
    volumes:
      - db_data:/var/lib/postgresql/data
  redis:
    image: redis:7-alpine
  volumes:
    db_data:
  networks:
    default:
      name: myapp_net
```



Run:

- 1) `docker compose build`
- 2) `docker compose up -d`
- 3) `docker compose ps`
- 4) `docker compose logs -f`

10) BEST PRACTICES

- ✓ Keep `compose.yml` in project root.
- ✓ Use meaningful service names.
- ✓ Use environment files for secrets & config.
- ✓ Use named volumes for persistent data.
- ✓ Use healthchecks for critical services.
- ✓ Use custom networks to isolate traffic.
- ✓ Keep images small & secure.
- ✓ Use profiles for optional/debug services.
- ✓ Use `docker compose config` to validate changes.



QUICK CHEAT SHEET

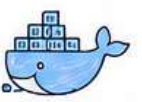
<code>up -d</code>	→ Start in background	<code>build</code>	→ Build images
<code>down</code>	→ Stop & remove	<code>pull</code>	→ Pull images
<code>ps</code>	→ List services	<code>config</code>	→ Show merged config
<code>logs -f</code>	→ Follow logs	<code>down -v</code>	→ Remove volumes too
<code>exec <svc> sh</code>	→ Shell in service		

SUMMARY: Docker Compose simplifies multi-container application management with a single YAML file and simple commands.
Master it to build, run, scale & maintain complex applications effortlessly!



7. REGISTRIES, DOCKER HUB, PRIVATE REGISTRIES, IMAGE OPTIMIZATION

Registries store & distribute Docker images.



1) WHAT IS A REGISTRY?

- A registry stores Docker images (repositories).
- Images are pushed to a registry and pulled from it.
- Public registries are open to everyone.
- Private registries are for organizations/teams.



2) TYPES OF REGISTRIES

Type	Examples	Description
Public	Docker Hub ghcr.io quay.io	Open to anyone. Free & paid plans. Community & official images.
Private	Harbor Nexus Repository AWS ECR GitLab Container Registry	For organizations. Access control, security, vulnerability scanning, replication.
Local (On-Prem)	Docker Registry (official registry:2)	Self-hosted registry inside your network.

3) DOCKER HUB

- Official public registry by Docker.
- Hosts Official, Verified Publisher & Community images.
- Rate limits for anonymous users.
- Create free account & authenticate.

Helpful Commands

```
docker login          # Login to Docker Hub
docker search nginx   # Search images
docker pull nginx:latest
docker push myuser/app:tag
docker logout
```

4) WORKING WITH DOCKER HUB

Login

```
docker login
Username: <your-username>
Password: *****
```

Tag Image

```
docker tag myapp:1.0 youruser/myapp:1.0
```

Push Image

```
docker push youruser/myapp:1.0
```

Pull Image

```
docker pull youruser/myapp:1.0
```

List Images

```
docker search <keyword>
docker pull <image>:<tag>
```

Image Naming Convention

```
<registry>/<namespace>/
<repository>:<tag>
```

Examples:

- nginx:latest
- youruser/app:1.0.0
- myregistry.local:5000/team/app:v2

If registry is omitted, Docker Hub is used by default.

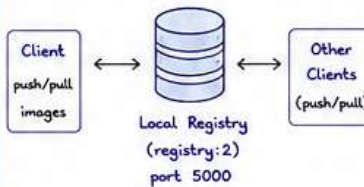
5) PRIVATE REGISTRIES

Why use?

- ✓ Security & access control
- ✓ Store internal/proprietary images
- ✓ No rate limits
- ✓ Compliance & auditing

Set up a local registry quickly:

```
docker run -d -p 5000:5000 --name registry \
-v /data/registry:/var/lib/registry registry:2
```



Tag & Push to Local Registry

```
docker tag myapp:1.0 localhost:5000/myapp:1.0
docker push localhost:5000/myapp:1.0
```

Accessing Private Registry

- By default, localhost:5000 is insecure (HTTP).
- For production, use HTTPS with valid certs.

Add to Docker daemon (Linux)

Edit /etc/docker/daemon.json

```
{
  "insecure-registries": ["registry.local:5000"]
}
```

Restart Docker:

```
sudo systemctl restart docker
```

Popular Private Registry Solutions

Solution	Features
Docker Registry (v2)	Lightweight, official, basic auth
Harbor	RBAC, vulnerability scan, UI, replication
AWS ECR	Managed, IAM integration
GitLab Container Registry	Integrated with GitLab
Nexus Repository	Multiple formats, enterprise

6) IMAGE TAGS & DIGESTS

- Tags are mutable (can change).
- Digests are immutable (content addressable).

Tags

```
myapp:latest myapp:1.0 myapp:v2.1
```

Digests (sha256)

```
myapp@sha256:3a7f...5e1d (unique)
```

Pull by digest for reliability:

```
docker pull nginx@sha256:3a7f...5e1d
```

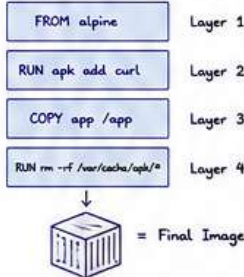
Get Image Digest

```
docker inspect --format="{{.RepoDigests}}" nginx
```

7) IMAGE OPTIMIZATION - BEST PRACTICES

- ✓ Use small base images (alpine, slim)
- ✓ Use multi-stage builds
- ✓ Minimize layers
- ✓ Remove unnecessary files & packages
- ✓ Use .dockerignore
- ✓ Combine RUN commands
- ✓ Use specific tags, avoid latest
- ✓ Clean up package manager cache

Each instruction adds a layer



Good ✓

FROM alpine:3.19	FROM ubuntu:latest
RUN apk add --no-cache \ curl git	RUN apk add curl
COPY . /app	COPY . /app
.dockerignore used	.git, node_modules in image
Multi-stage build	Single-stage build
Specific tag (1.2.3)	Using latest tag

Bad ✗

8) .DOCKERIGNORE FILE

Exclude files/folders from build context.

Example .dockerignore

```
.git
node_modules
*.log
.env
Dockerfile
docker-compose.yml
README.md
```

Benefit

- Smaller build context
- Faster builds
- Smaller images
- Better security



9) IMAGE SIZE ANALYSIS

Analyze image size & layers.

Commands:

```
docker images
docker history <image>
dive <image> # (dive tool)
```

```
$ docker history myapp:1.0
IMAGE      CREATED      SIZE      COMMENT
abc123     2 min ago   5MB       RUN apk add curl
...
```



Use dive for interactive layer analysis.

10) SECURITY & SCANNING

- Scan images for vulnerabilities before pushing.
- Use official or trusted base images.
- Keep base images up-to-date.
- Don't store secrets in images.

Tools for scanning:



Docker Scout (Docker Hub)



Trivy



Grype



Snyk

11) IMAGE LIFECYCLE IN REGISTRY



SUMMARY

- ★ Use registries to store & distribute images.
- ★ Docker Hub for public, Private/Local registries for private.
- ★ Tag images properly, prefer immutable digests.
- ★ Optimize images for size, speed & security.
- ★ Scan & update images regularly.



8. DOCKER SECURITY (Stay Safe, Ship Secure!)



Security is not a feature, it's a practice.

1) SECURITY PRINCIPLES

- Least Privilege: give only what is needed.
- Defense in Depth: multiple layers of protection.
- Immutable Infrastructure: rebuild, don't patch.
- Keep it Updated: images, engine, host.
- Secure by Default: deny by default.

Key Areas to Secure



- Images
- Containers (runtime)
- Secrets & Configs
- Host & Daemon
- Networks
- Volumes & Data
- Supply Chain
- Access & Users

2) LEAST PRIVILEGE CONTAINERS

Run containers with minimal privileges.

Run as non-root user:

```
docker run -u 1000:1000 myapp
```

Drop Linux capabilities:

```
docker run --cap-drop=ALL \
```

```
--cap-add=NET_BIND_SERVICE myapp
```

Read-only root filesystem:

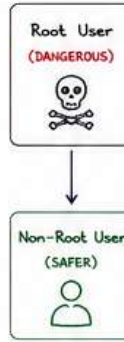
```
docker run --read-only myapp
```

Restrict resources:

```
docker run -m 512m --cpus=0.5 myapp
```

No new privileges:

```
docker run --security-opt no-new-privileges myapp
```



3) IMAGES SECURITY BEST PRACTICES

- Use official & minimal base images (alpine, distroless).
- Scan images for vulnerabilities.
- Keep images up-to-date.
- Remove unnecessary packages & files.
- Use multi-stage builds.
- Don't store secrets in images.
- Pin base image versions (avoid latest).

Good Base Images

- ✓ alpine
- ✓ distroless
- ✓ scratch
- ✓ ubi-minimal

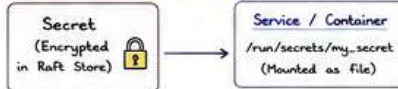
Bad Practices

- ✗ Using latest
- ✗ Huge base images
- ✗ Untrusted images
- ✗ Secrets in image

4) SECRETS MANAGEMENT

- Never hardcode secrets in images or Dockerfiles.
- Use Docker Secrets (in Swarm) or external secret stores.
- Pass secrets at runtime.

Docker Secrets (Swarm mode)



Commands:

```
echo "mysecret" | docker secret create my-secret -
docker service create --name web \
--secret my-secret nginx
# Inside container: cat /run/secrets/my-secret
```

Other Secret Options

- Env vars at runtime: `-e DB_PASSWORD=***` (beware logs)
- Docker Compose: use secrets (file based)
- External: HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, GCP Secret Manager

5) ACCESS CONTROL & USERS



- Restrict access to Docker daemon.
- Use Docker groups wisely.
- Prefer rootless Docker.
- Use role-based access in registries.

Users & Permissions

User / Group	Access Level
root	Full access (all powerful)
docker group	Full access to Docker daemon (Essentially root)
Regular user	No Docker access
Rootless user	Limited to user namespace (Safer)

Best Practices

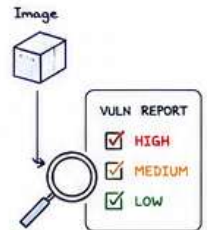
- ✓ Avoid running Docker daemon as root (use rootless).
- ✓ Don't add unnecessary users to docker group.
- ✓ Use RBAC in registries (Harbor, GitLab, ECR).
- ✓ Use 2FA for registry access where possible.

6) SCANNING IMAGES FOR VULNERABILITIES

Scan early, scan often!

Popular Tools:

- Docker Scout (Docker Hub)
- Trivy (Aqua Security)
- Gype
- Snyk
- Anchore



Example with Trivy:

```
trivy image nginx:1.25
```

Fail build on critical vulns (CI/CD):

```
trivy image --exit-code 1 --severity CRITICAL nginx:1.25
```

Docker Scout (built-in):

```
docker scout quickview nginx:1.25
```

7) ROOTLESS DOCKER

- Run Docker daemon without root privileges.
- Reduces risk of daemon compromise.
- Uses user namespaces & cgroups v2.

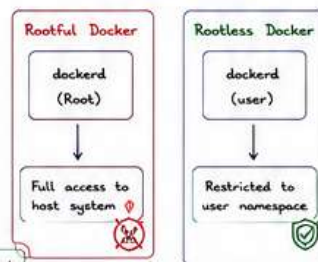
Enable Rootless Docker

```
# Install (Ubuntu example)
sudo apt install -y uidmap dbus-user-session
dockerd-rootless-setup tool.sh install
```

Use

```
export DOCKER_HOST=unix://$XDG_RUNTIME_DIR/docker.sock
docker info
```

Rootful vs Rootless



Limitations

- Some features (overlays, ports <1024) may be restricted.
- Less impact if daemon is compromised.

8) RUNTIME SECURITY

- Apply security profiles (AppArmor, SELinux, Seccomp).
- Use seccomp to restrict syscalls.
- Block privilege escalation.
- Monitor containers at runtime.

Security Profiles

- AppArmor (default on many distros)
- SELinux (enforcing mode recommended)
- Seccomp (default: docker/default.json)

Use custom seccomp profile:

```
docker run --security-opt seccomp=./seccomp.json myapp
```

What to Avoid

- ✗ `--privileged`
- ✗ `--pid=host`
- ✗ `--network=host` (unless needed)
- ✗ Sharing host namespaces
- ✗ Mounting sensitive host paths (unless required)

Check container security

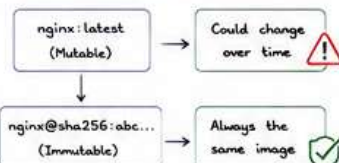
```
docker inspect --format='{{.json.HostConfig}}' myapp |
jq '.SecurityOpt, .CapAdd, .CapDrop'
```

9) SUPPLY CHAIN SECURITY

- Verify & trust what you build and deploy.
- Use minimal, trusted base images.
- Pin image digests (immutable).
- Sign images.
- Scan dependencies (packages, libraries).

Use Image Digests (instead of tags)

```
docker pull nginx@sha256:abcdef123456...
```



Sign Images (Docker Content Trust/Cosign)

Cosign Example:

```
cosign generate-key-pair
cosign sign myapp:1.0
cosign verify myapp:1.0
```

Enable Content Trust (Notary v1)

```
export DOCKER_CONTENT_TRUST=1
docker push myapp:1.0
```

Other Best Practices

- Verify upstream sources.
- Scan base images.
- Use SBOM (Software Bill of Materials).
- Automate checks in CI/CD pipeline.

10) HOST & DAEMON SECURITY

- Keep Docker & host OS updated.
- Restrict Docker socket access.
- Use TLS for remote Docker API.
- Audit & monitor Docker activity.

Protect the Docker Socket

```
# Bad (world writable - DANGEROUS)
ls -l /var/run/docker.sock

# Good (restricted)
sudo chown root:docker /var/run/docker.sock
sudo chmod 660 /var/run/docker.sock
```

Audit & Monitoring

- Enable daemon logs: `/var/log/docker.log`
- Monitor with tools: Falco, Docker Audit, Sysdig
- Alert on suspicious activity.

Enable TLS for Remote API

```
dockerd --tlsverify \
--tlscacert=ca.pem \
--tlscert=server-cert.pem \
--tlskey=server-key.pem \
-H tcp://0.0.0.0:2376
```

Host Hardening Checklist

- ✓ Firewall enabled
- ✓ Disable unused services
- ✓ Use up-to-date kernel
- ✓ Apply patches regularly
- ✓ Use strong SSH keys
- ✓ Limit root access
- ✓ Regular backups

SUMMARY



- ✓ Follow least privilege at every layer.
- ✓ Scan images & fix vulnerabilities.
- ✓ Use rootless Docker when possible.
- ✓ Manage secrets securely.
- ✓ Secure the host, daemon & network.
- ✓ Automate security in CI/CD.

Secure today, ship safely tomorrow!

9. DOCKER IN DEVOPS (CI/CD, DEPLOYMENTS, MONITORING, LOGGING)

Automate everything, monitor what matters, ship with confidence!



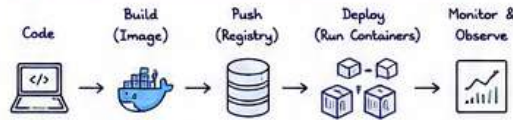
1) DOCKER IN DEVOPS - WHY?

- Consistent environments (dev → test → prod)
- Faster builds & deployments
- Isolation & resource efficiency
- Easy rollback & version control
- Works everywhere (laptop → cloud → prod)

Core Benefits

- ✓ Portability
- ✓ Scalability
- ✓ Reliability
- ✓ Faster Delivery
- ✓ Better Resource Utilization
- ✓ Simplified Operations

2) TYPICAL DEVOPS WORKFLOW WITH DOCKER



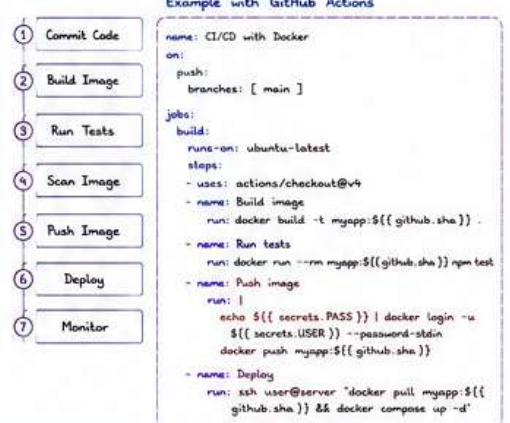
Key Components

- Dockerfile → Build image
- Registry → Store & distribute
- Container → Run app
- Orchestrator → Manage at scale (Compose, Swarm, K8s)
- Monitoring → Health, metrics, alerts
- Logging → Centralized logs

Popular Stack

- Git + CI (Jenkins/GitHub Actions/GitLab CI)
- Docker + Registry (Hub/ECR/GCR)
- Orchestrator (Compose/K8s/Swarm)
- Monitoring (Prometheus + Grafana)
- Logging (ELK/EFK Stack)

3) CI/CD PIPELINE EXAMPLE



4) BUILD & PUSH IMAGE (IN PIPELINE)

Build:

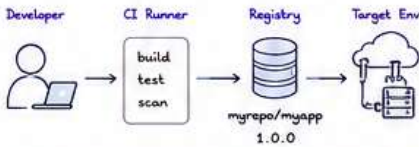
```
docker build -t myapp:1.0.0 .
```

Tag:

```
docker tag myapp:1.0.0 myrepo/myapp:1.0.0
```

Push:

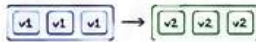
```
docker push myrepo/myapp:1.0.0
```



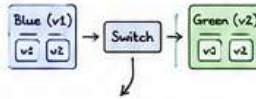
Use immutable tags (1.0.0, build-id, commit-sha)
Avoid :latest in pipelines.

5) DEPLOYMENT STRATEGIES

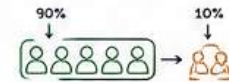
- Rolling Update
Update containers one by one.



- Blue/Green Deployment
Switch traffic between environments.



- Canary Deployment
Release to a small % of users first.



Docker Compose Deployment

1. Pull new images
2. Recreate containers
3. Healthcheck
4. Remove old containers

Example:

```
docker compose pull
docker compose up -d
--remove-orphans
```

Rollback

```
docker compose down
docker compose up -d
--no-deps old_service
```

6) ORCHESTRATION OPTIONS

Tool	Use Case	Key Features
Docker Compose	Single host, dev/test setups	Simple YAML, easy to use, supports networks & volumes
Docker Swarm	Multi-host clustering	Built-in, lightweight, service discovery, rolling update
Kubernetes	Large scale, production	Auto-scaling, self-healing, rich ecosystem
ECS / ACI / GKE	Managed Services	Fully managed, integrates with cloud

When to use?

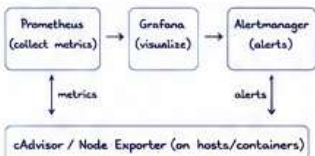
- Compose → local dev, small apps
- Swarm → simple clustering
- Kubernetes → complex, large-scale apps
- Managed Services → less ops overhead



7) MONITORING CONTAINERS

- Monitor health, performance, availability.
- Detect issues early.
- Key Metrics: CPU, Memory, Network, Disk, Restart count.

Tools Stack



Monitor with CLI

```
# Container stats
docker stats

# Container info
docker inspect <container-id>

# Health status
docker ps --format '{{.Names}}\t{{.Status}}'
```



8) LOGGING & CENTRALIZED LOGS

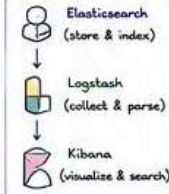
- Containers are ephemeral - logs go away.
- Centralize logs for search, alerting & audit.

Logging Drivers (built-in)

- json-file (default)
- syslog
- journald
- fluentd
- gelf
- awslogs
- splunk

```
Set log driver:
docker run --log-driver=fluentd \
--log-opt fluentd-address=localhost:24224 myapp
```

ELK Stack (Popular)



View logs:

```
docker logs -f <container>
docker compose logs -f
```

9) HEALTHCHECKS

- Check if containerized app is healthy.
- Docker can auto-restart unhealthy containers.

In Dockerfile

```
HEALTHCHECK --interval=30s --timeout=5s \
--retries=3 CMD curl -f http://localhost/health \
|| exit 1
```

In docker-compose.yml

```
services:
  web:
    image: myapp
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost/health"]
      interval: 30s
      timeout: 5s
      retries: 3
```

Check status:

```
docker inspect --format='{{.State.Health.Status}}' <container>
```

10) ENVIRONMENT MANAGEMENT

- Use environment variables & secrets.
- Separate configs per environment.

Example .env

```
DB_HOST=db
DB_USER=appuser
DB_PASS=secret123
APP_ENV=production
```

In Compose

```
services:
  app:
    env_file:
      - .env
    environment:
      - TZ=UTC
```

Best Practices

- ✓ Don't commit secrets
- ✓ Use Docker secrets or external secret manager (Vault, AWS SM)
- ✓ Use .env for non-sensitive config

11) BACKUP & DATA MANAGEMENT

- Containers are ephemeral, data must persist.
- Use volumes & external storage.
- Backup databases & volumes regularly.

Example (backup volume data)

```
docker run --rm -v mydata:/data -v $(pwd):/backup \
alpine tar czf /backup/data-backup-$(date +%F).tgz -C /data .
```

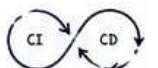
Restore:

```
docker run --rm -v mydata:/data -v $(pwd):/backup \
alpine tar xzf /backup/data-backup-2024-05-20.tgz -C /data
```



12) BEST PRACTICES

- ✓ Automate builds, tests, scans in CI.
- ✓ Use small, secure base images.
- ✓ Tag images with version/commit SHA.
- ✓ Scan images for vulnerabilities.
- ✓ Use healthchecks.
- ✓ Centralize logs & monitor everything.
- ✓ Use infrastructure as code (Compose/Terraform).
- ✓ Don't run containers as root.
- ✓ Keep dev, test, prod parity.
- ✓ Document & version your Dockerfiles & configs.



SUMMARY



Docker supercharges DevOps by enabling consistent builds, reliable deployments, continuous delivery, observability and secure operations.

Build → Test → Scan → Push → Deploy → Monitor
Automate | Observe | Secure | Improve



10. DOCKER IN DEVOPS (CI/CD, DEPLOYMENTS, MONITORING, LOGGING)

Observe, automate & iterate
Ship with confidence!



1) DOCKER IN DEVOPS - BEST PRACTICES

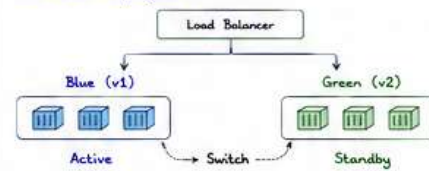
- ✓ Keep Dockerfiles small & simple.
- ✓ Use official base images.
- ✓ Pin image versions (avoid :latest).
- ✓ Run as non-root user.
- ✓ Use .dockerignore to reduce build context.
- ✓ Scan images for vulnerabilities.
- ✓ Store secrets in env/secrets manager.
- ✓ Use multi-stage builds.
- ✓ Keep containers stateless.
- ✓ Automate everything in CI/CD.

Motto: Build once → Ship anywhere → Run reliably

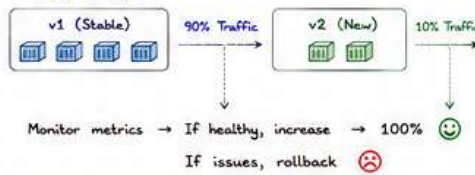


2) BLUE/GREEN & CANARY DEPLOYMENT (WITH DOCKER)

Blue/Green Deployment



Canary Deployment



3) DOCKER COMPOSE FOR ENVIRONMENTS

compose.yml (example)

```
version: "3.9"
services:
  web:
    image: myapp:1.0.0
    ports:
      - "8080:80"
    depends_on:
      - db
    environment:
      - DB_HOST=db
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret
    volumes:
      - db_data:/var/lib/postgresql/data
volumes:
  db_data:
```

Use profiles for different envs.

Profiles Example

- dev
- staging
- prod

Commands

```
docker compose up -d
docker compose down
docker compose ps
docker compose logs -f
docker compose exec <svc> sh
```

4) CI/CD PIPELINE STAGES (WITH DOCKER)



1. Code - Developer commits code to VCS (Git).
2. Build - Docker image built from Dockerfile.
3. Test - Unit/Integration tests inside container.
4. Scan - Vulnerability scan (Trivy/Grupe/Snyk).
5. Push - Push image to registry (Docker Hub/Private).
6. Deploy - Deploy to environment (K8s/Swarm/VM/Cloud).
7. Monitor - Monitor health, logs, metrics.

Popular CI/CD Tools

- GitHub Actions
- GitLab CI/CD
- Jenkins
- CircleCI
- Azure DevOps
- Argo CD

5) DOCKER SWARM STACK DEPLOYMENT

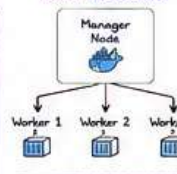
Deploy a stack:

```
docker stack deploy -c stack.yml myapp
```

stack.yml (simplified)

```
version: "3.9"
services:
  web:
    image: myapp:1.0.0
    replicas: 3
    ports:
      - "80:80"
  db:
    image: postgres:15
    volumes:
      - db_data:/var/lib/postgresql/data
volumes:
  db_data:
```

Swarm Architecture

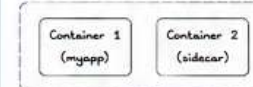


Swarm Commands

```
docker node ls
docker service ls
docker service ps <service>
docker stack ls
docker stack rm <stack>
```

6) KUBERNETES WITH DOCKER IMAGES

Pod (runs containers)

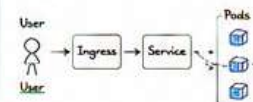


Key Objects

- Pod
- Deployment
- Service
- ConfigMap
- Secret
- Ingress

Kubernetes Flow

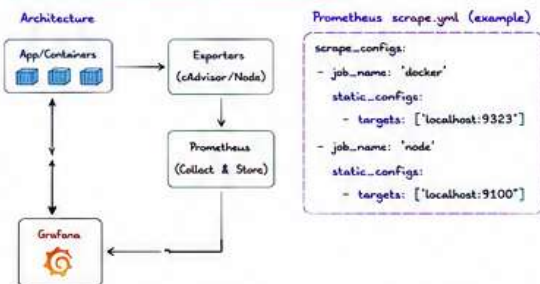
1. Image pushed to registry
2. Kubernetes pulls image
3. Pod created
4. Service exposes app
5. Ingress routes traffic



kubectl useful

```
kubectl get pods
kubectl get svc
kubectl describe pod <pod>
kubectl logs <pod>
kubectl rollout status deploy/<name>
```

7) PROMETHEUS + GRAFANA MONITORING



Useful Metrics

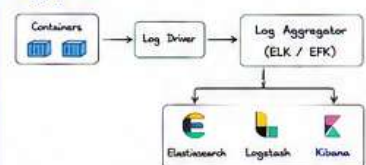
- container_cpu_usage_seconds_total
- container_memory_usage_bytes
- container_network_receive_bytes_total
- container_network_transmit_bytes_total

Grafana Tips

- ✓ Use dashboards (cAdvisor, Node Exporter)
- ✓ Set alerts (CPU, Memory, Down)
- ✓ Visualize trends & capacity

8) CENTRALIZED LOGGING

Logging Flow



Docker log drivers (built-in)

- json-file (default)
- syslog
- journald
- gelf
- fluentd
- awslogs
- splunk

EFK Stack

- Elasticsearch - store & index
- Fluentd - collect & parse
- Kibana - visualize & search

Attach gelf driver (example)

```
docker run --log-driver=gelf \
--log-opt gelf-address=udp://loghost:12201 myapp
```

9) PIPELINE EXAMPLE (GITHUB ACTIONS)

.github/workflows/ci-cd.yml

```
name: CI/CD with Docker
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build image
        run: docker build -t myapp:${{ github.sha }} .
      - name: Login to Registry
        run: echo ${{ secrets.PASS }} | docker login -u ${{ secrets.USER }} --password-stdin
      - name: Push image
        run: docker push myregistry/myapp:${{ github.sha }}
  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to Server
        run: ssh user@server 'docker pull myregistry/myapp:${{ github.sha }} && docker compose up -d'
```

Secrets Used

- secrets.USER
- secrets.PASS
- secrets.SSH_KEY

Best Practices

- ★ Trigger on push/tag
- ★ Cache dependencies
- ★ Scan before push
- ★ Use environments (staging/prod)
- ★ Requires approvals

10) TROUBLESHOOTING DOCKER CONTAINERS

Problem	Check / Command	What to Look For
Container not starting	docker logs <container>	Error logs, missing deps, wrong config
High CPU usage	docker stats	CPU hogging container
High Memory usage	docker stats	Memory leak / limits exceeded
Port not reachable	docker port <container>	Port mapping, firewall rules
Network issues	docker network inspect <net>	IP conflicts, wrong network
Image pull failed	docker pull <image>	Auth, network, registry
Disk full	docker system df	Clean up images, volumes, logs

Cleanup Commands

```
docker system prune -a
docker volume prune
docker image prune -a
docker container prune
```

Observe → Diagnose
→ Fix → Improve



11) INFRASTRUCTURE AS CODE (IaC) WITH DOCKER

- Use IaC to provision & manage Docker environments.
- Tools: Terraform, Ansible, Pulumi, CloudFormation.
- Docker provider example (Terraform):

```
provider "docker" {}

resource "docker_image" "app" {
  name = "myapp:1.0.0"
}

resource "docker_container" "app" {
  name = "myapp-container"
  image = docker_image.app.name
  ports {
    internal = 80
    external = 8080
  }
}
```

- ✓ Store state remotely
- ✓ Use versioning control
- ✓ Plan → Apply → Verify

12) CONTINUOUS IMPROVEMENT CHECKLIST

- ✓ Automate builds, tests, scans & deployments.
- ✓ Keep base images updated.
- ✓ Pin versions, avoid :latest.
- ✓ Use smallest secure images.
- ✓ Scan regularly for vulnerabilities.
- ✓ Monitor health, logs & metrics.
- ✓ Centralize logs & alerts.
- ✓ Back up data & configs.
- ✓ Practice rollback & disaster recovery.
- ✓ Document everything.



DevOps is a journey, not a destination.
Automate, observe & keep improving!



SUMMARY: Docker empowers DevOps teams to build, ship & run applications consistently.



Automate today,

Deliver better tomorrow!

